ICEE
Intelligent Computing for Energy and Environment

Research Article | Intelligent Computing for Energy and Environment | Vol. 1, No. 1 (2026) | DOI: XX.XXXXX/icee.202X.XXXX

AI-Powered Intrusion Detection for Programmable Logic Controllers in Smart Grid and Industrial Energy Networks: A Reproducible Docker-Based Simulation Platform

Natali Soukh[1], **Zainab Taha**[1], **and Passent Elkafrawy** *[1]

[1]*Department of Computer Science, Effat University, Jeddah, Saudi Arabia*

Corresponding author: *Passent Elkafrawy | corresponding@effat.edu.sa | https://orcid.org/0000-0002-8557-4286*

Received 20 August 2026 **Accepted** 13 October 2026 **Published** 13 October 2026

Handling Editor Badre Bossoufi, and Mohamed El-Amin

ABSTRACT

Operational Technology (OT) networks in electrical grids, substations, and other energy-sector critical infrastructure are increasingly interconnected with corporate IT networks under Industry 4.0, exposing legacy control protocols to cybersecurity risks that were never considered in their original design. Commercial OT security platforms offer enterprise-grade monitoring but remain closed-source, costly, and inaccessible for reproducible research, while academic work tends to validate isolated algorithms without a complete, testable platform. This paper presents a fully containerized, Docker-based simulation of an enterprise/OT network – modelled on the IT/OT segmentation typical of substation and industrial energy environments – that generates its own labelled traffic, trains a Random Forest classifier on 22 flow-level features, and performs live detection through a Streamlit dashboard. The pipeline combines four deterministic threshold rules with a supervised machine-learning layer to detect a simulated two-stage attack (reconnaissance followed by data exfiltration). Five-fold cross-validation of the Random Forest model achieved 100% ± 0.000 accuracy on the internally generated dataset, with complete separation between normal (0.000–0.015) and attack (0.950–1.000) probability scores. Feature-importance analysis identified cross-subnet traversal, flow duration, and TCP SYN count as the strongest discriminators of malicious activity in this synthetic setting. The platform is open, replicable by anyone with access to Docker, and is proposed as a low-cost research and training testbed for intrusion detection in energy-sector OT networks, with extension to real PLC protocol traffic and public benchmark validation planned as a second project phase hello.

Keywords: *Industrial Control Systems; Smart Grid Security; Intrusion Detection; Machine Learning; Programmable Logic Controllers; Critical Infrastructure Protection; Random Forest*

1 Introduction

Critical energy infrastructure – electrical grids and substations, water treatment plants, and manufacturing and transportation systems – depends on Industrial Control Systems (ICS) and Programmable Logic Controllers (PLC) are a key component. Traditionally, PLCs have been air-gapped, meaning they did not connect to other networks. In today's Industry 4.0, however, Operational Technology (OT) networks are regularly interconnected with corporate IT networks and, by extension, to the internet, creating an environment in which legacy PLC protocols must contend with cybersecurity risks not considered during their original design.

This exposure carries energy-system stakes that go beyond conventional IT security. A successful intrusion into a substation or grid-control network can disrupt supervisory control and data acquisition (SCADA) functions, trigger cascading outages, and compromise the reliability of electricity supply to end users –

consequences with direct safety and economic impact rather than mere data loss. Securing PLC-based OT networks is therefore inseparable from the broader goal of grid resilience.

Conventional IT-based security measures such as deep packet inspection firewalls, antivirus engines, and heavy anomaly detectors cannot function efficiently within a PLC setting due to three primary factors:

- Time-criticality is strictly defined in milliseconds.
- Central processing capabilities on the device itself are limited.
- Deterministic timing requirements are such that even a slight delay created by security software may lead to disastrous results.

Another gap which goes unnoticed is the lack of accessible, replicable research tools. While off-the-shelf OT security products available from companies like Nozomi Networks, Claroty, and Dragos provide advanced enterprise-wide monitoring capabilities, they are closed-source and proprietary – offering no access to the machine-learning pipeline, no capability to generate datasets, and no ability to simulate attacks safely. Research from academic institutions has produced promising individual algorithms but lacks a complete platform.

In filling both gaps, this research project proposes a complete solution in the form of a fully contained Docker-based simulation of an enterprise/OT network, representative of the IT/OT segmentation found in substation and industrial energy environments, that can be used for IDS research. The proposed research platform generates its own labelled datasets and uses them to train a Random Forest algorithm while simultaneously performing detection via a Streamlit live dashboard. The current implementation operates at the enterprise/OT network-flow level; extension to native PLC protocol traffic (Modbus, DNP3) is addressed as future work in Section 4.7)

2 Literature Review

This section reviews literature related to intrusion detection, the implementation of AI, and PLCs, in order to situate the unique contributions of the current project – including work specific to smart-grid and SCADA cybersecurity.

2.1 Edge and On-Device AI for PLCs

Jordan [1] shows that lightweight AI algorithms can be run on PLCs themselves via Arduino Opta boards, resulting in low-latency anomaly detection without any information being transferred from the device. Although useful for secure systems, this approach relies on certain PLC architectures and has limitations in terms of algorithm depth. Körösi et al. [2] expand upon this work, showing that deep-learning inference can be containerised within Docker services on PLCs without requiring AI accelerators, though computational power on non-specialized PLCs remains a concern.

Rodriguez, Dhameliya, and Patel [3] explore the wide range of practical uses of AI for PLC automation, including predictive maintenance, optimization, and pattern recognition. However, they discuss the potential benefits theoretically without experimental evidence.

2.2 AI-Based Intrusion Detection in ICS Networks

Ali et al. [4] address the issue of multi-class imbalance in ICS datasets through big-data methodologies aimed at boosting the recall of minority classes representing attacks within ICS networks – a practical concern for realistic traffic capture where attack flows are inherently infrequent.

The idea of splitting anomaly detection into two independent unsupervised models dedicated to each of the traffic regimes is introduced by Elnour et al. [5], and it directly inspired the dual-layer detection approach followed in the current work. Elsaid and Binbusayyis [6] focus on optimizing Isolation Forest for the analysis of heterogeneous IIoT data streams, while Boateng and Bruce [7] analyze unsupervised ensembles for PLC-based process control, showing that unsupervised anomaly detection alone generates false positives.

Gawehn, Ergenc, and Fischer [8] use deep learning across several PLCs at once and show how anomaly detection can be performed across devices, provided there is proper alignment of features. Mitsubishi Electric and Nozomi Networks [9] have deployed AI-equipped sensors in actual PLCs, and Schneider Electric and Siemens have introduced AI co-pilots within PLC programming software to reduce the risk of human coding errors.

2.3 Cybersecurity in Smart Grids and SCADA Systems

A growing body of work applies machine learning specifically to smart-grid and SCADA cybersecurity. Sahani et al. [10] survey machine-learning-based intrusion detection for smart-grid computing, cataloguing the distinct threat surfaces of Advanced Metering Infrastructure (AMI), substation automation, and SCADA communication layers, and note that most proposed detectors are validated on synthetic or simulated power-system data rather than deployed platforms. Alrumaih and Alenazi [11] propose a centrality- and graph-aware deep-learning model for detecting cyberattacks targeting ICS within critical infrastructure, emphasizing topology-aware features – a direction consistent with the cross-subnet traversal signal used in the present work. Rabie et al. [12] propose a stochastic neural-network security model for smart-grid SCADA systems, while Mohan, Ravikumar, and Govindarasu [13] present a distributed, semantic-rule-based intrusion detection system for SCADA in smart grids, combining rule-based and learned detection layers in a manner conceptually similar to the hybrid rule-plus-ML approach adopted here. Collectively, this literature confirms that grid-specific IDS research is active but, as in general ICS security research, is rarely accompanied by an openly reproducible simulation platform.

2.4 The Gap: No Reproducible Hybrid Platform

Based on this literature synthesis, two groups can be identified: commercial off-the-shelf security solutions for OT that work well but are costly and not easy to understand or extend; and academic projects that refine individual models but do not provide a unified, reproducible framework covering data generation, detection, and visualization together. This project addresses that gap. Table 1 contrasts the proposed platform with three leading commercial OT security products.

Table 1. Competitor feature comparison.

Feature	Nozomi Networks	Claroty	Dragos	This Project
ML-driven anomaly detection	✓	✓	✓	✓
PLC-specific simulation & testing environment	—	—	—	✓
Lightweight / cost-effective for SMEs	—	—	—	✓
Access to ML pipeline & dataset generation	—	—	—	✓
Dual ML: supervised + unsupervised	Proprietary	Proprietary	Proprietary	✓
Full pipeline (traffic → detect → report)	—	—	—	✓

3 Methodology

The methodology establishes an IDS pipeline using a Docker network simulation, whereby the data used to train the Random Forest classifier are generated internally within the environment and tested in real time as the IDS captures new traffic. The sub-sections below describe the lab environment, dataset, and visualizations produced.

3.1 Data Information

The training dataset used was entirely generated internally from the network simulation. No external or publicly available datasets were used to train the classifier. This is by design: it ensures that the traffic dataset accurately represents the protocols' behavior and timing distribution of the network environment.

3.1.1 Network Architecture

The lab runs nine Docker containers across two isolated bridge networks, structured to mirror the control-LAN / corporate-LAN segmentation typical of substation and industrial energy environments:

- **Enterprise network** – subnet 10.10.1.0/24 – contains all internal servers and workstations.
- **Attacker network** – subnet 10.20.1.0/24 – contains only the red-team attacker machine.

The IDS container has one network interface in each subnet, making it a bridge through which all packets must flow. This gives the IDS visibility into all traffic passing between zones without altering the application layer of any other container. Table 2 lists all container roles and IP assignments.

Table 2. Container hosts, network interfaces, and roles in the Docker-based lab environment.

IP Address	Hostname	Zone	Role
10.10.1.10	web-server	DMZ	Apache + PHP web server (HTTPS, port 443)
10.10.1.20	db-server	DMZ	MySQL 8.0 database server
10.10.1.31	hr-pc	Internal	HR workstation – HTTPS browsing, ICMP pings
10.10.1.32	it-pc	Internal	IT workstation – monitors all hosts, checks DB
10.10.1.33	finance-pc	Internal	Finance workstation – HTTPS browsing
10.10.1.50	ids	Enterprise NIC	IDS internal-facing interface
10.20.1.10	ids	Attacker NIC	IDS attacker-facing interface
10.20.1.100	attacker	Red Team Net	Simulated attacker – full kill chain

All containers route their default gateway through the IDS. A packet from the attacker (10.20.1.100) destined for the web server (10.10.1.10) travels: attacker → IDS (10.20.1.10) → IDS (10.10.1.50) → web-server. The return path is symmetric, so the IDS captures both sides of every conversation.

3.1.2 Traffic Generation

Two Python scripts produce the traffic used for training and live evaluation:

- `generate_normal_traffic.py`: simulates realistic employee activity over 25 capture cycles – HTTPS logins and portal browsing from HR, Finance, and IT workstations; ICMP pings from the IT workstation to all hosts; MySQL queries from the web server to the database backend.
- `generate_attacker_traffic.py`: simulates a two-stage red-team intrusion over 25 cycles – (1) Reconnaissance: ICMP ping sweep, UDP port probes, nmap TCP SYN scan; (2) Data Exfiltration: direct MySQL dump of the employee table, HTTPS scraping of all portals, hping3 SYN burst.

3.2 Dataset Description

The IDS pipeline collects packets every cycle for 10 s using `tcpdump` operating in promiscuous mode inside the IDS Docker container. Packets from the same source IP are combined into flows, and 22 numeric features are computed for each flow using Scapy. The resulting CSV files form the training set.

3.2.1 Dataset Statistics

Table 3 summarizes the composition of the training dataset, including the label distribution, flow counts, and the script responsible for generating each traffic category.

Table 3. Dataset statistics for the internally generated training set.

Split	Label	Flows	Proportion	Source Script
Training – normal	0	855	89.1%	`generate_normal_traffic.py`
Training – attack (recon)	1	75	7.8%	`generate_attacker_traffic.py`
Training – attack (exfil)	1	30	3.1%	`generate_attacker_traffic.py`
Total	—	**960**	**100%**	-

The skew (approximately 8:1 normal to attack) is by design and reflects reality, as any healthy corporate or OT network will show very little attack activity. Nonetheless, the model achieves complete separation, thanks to 22 attributes that are practically orthogonal across classes – subnet crossing, SYN flooding, and unusual protocol combinations. The 105 attacks are further split into 75 recon scans (~71%) and 30 data exfiltrations (~29%).

3.2.2 Feature Set (22 Numeric Features)

Table 4 groups the 22 numeric features computed per flow.

Table 4. Feature groups and the 22 numeric features computed per flow.

Group	Features
Volume	`bytes_total`, `connection_count`, `pkt_rate_per_sec`, `flow_duration_sec`
Connection	`tcp_syn_count`, `unique_hosts`, `unique_ports`, `cross_subnet`
ICMP	`icmp_count`, `icmp_request_count`
Protocol counts	`port_ssh_count`, `port_telnet_count`, `port_smtp_count`, `port_dns_count`, `port_http_count`, `port_https_count`, `port_smb_count`, `port_mysql_count`, `port_pgsql_count`, `port_rdp_count`, `port_winrm_count`
Other	`udp_count`

3.2.3 Feature Engineering and Reduction

No external dimensionality-reduction method such as Principal Component Analysis (PCA) was applied. Prior to training, the full 22-feature set is standardized via the `StandardScaler` transformation within the scikit-learn `Pipeline`, giving features unit variance for the Random Forest algorithm. Feature selection is then conducted automatically by the Random Forest via Gini importance. The five highest-importance features – `cross_subnet`, `flow_duration_sec`, `tcp_syn_count`, `unique_ports`, and `port_ssh_count` – contribute approximately 71% of total feature importance.

3.2.4 Regularisation and Data Augmentation

The training pipeline uses two explicit forms of regularisation. First, all 22 features are scaled using scikit-learn's `StandardScaler` prior to classification, preventing high-magnitude features such as `bytes_total` from dominating low-magnitude ones such as `cross_subnet` (binary 0/1). Second, the Random Forest classifier is configured with `min_samples_leaf=2`, forcing each leaf node to contain at least two training instances and thereby limiting memorization of noise in individual flows. The ensemble of 100 trees (`n_estimators=100`, `max_depth=None`, `random_state=42`) further reduces variance via bagging.

Data augmentation techniques such as SMOTE were not employed, given the moderate class imbalance (8:1) and the near-orthogonal separation between attack and normal flows; cross-validation accuracy of 100% ± 0.000 indicates that additional synthetic minority samples would provide no benefit.

3.3 Visualisations

The following visualisations were produced from the generated dataset and are referenced throughout the analysis.

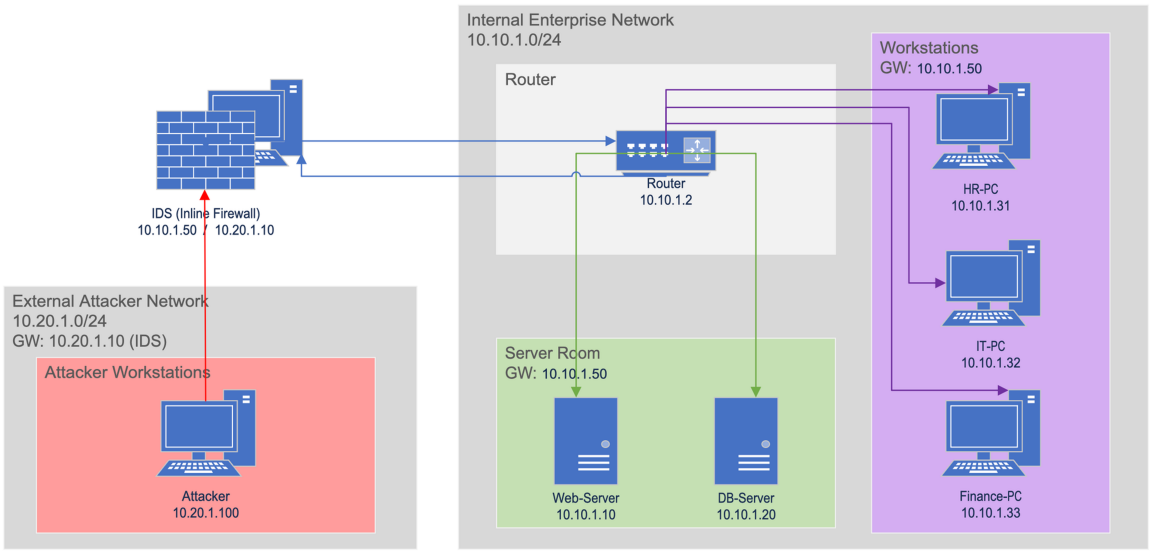


Figure 1. Network topology diagram. Docker bridge networks isolated from each other (10.10.1.0/24 – enterprise, 10.20.1.0/24 – attacker), joined via the IDS container running as an inline bridge, showing all eight hosts with their IPs and packet-flow routing through the IDS.

Figure 2 summarises the severity levels assigned by the rule-based layer based on traffic-pattern thresholds; the highest-priority trigger within a flow determines its severity.

Severity	What Triggered It	Metric Threshold
Critical	Large data transfer / attacker reaching internal systems	> 1 MB transferred, or known attacker IP contacting internal target
High	Active port scanning	> 20 TCP SYN packets in one window
Medium	Ping sweep / unusual port access	> 10 ICMP requests or > 5 unique destination ports
Low	Attacker-flagged host, no specific attack behavior	Known attacker IP, but thresholds not yet crossed
Normal	No suspicious behavior detected	No tags triggered

Figure 2. IDS alert severity levels. Requirements for each severity level (CRITICAL, HIGH, MEDIUM, LOW, NORMAL) in terms of traffic-pattern thresholds; severities are assigned based on the highest-priority detection trigger within a flow.

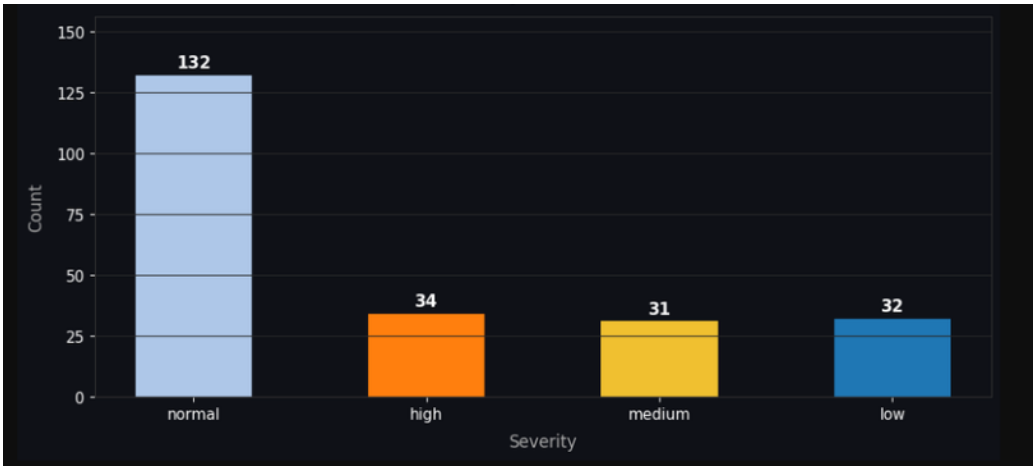


Figure 3. Severity breakdown across all 229 intercepted flows, showing the distribution of NORMAL, MEDIUM, HIGH, and CRITICAL severity levels; the majority of detected flows are NORMAL.

Note that Table **??** describes the *training* dataset (960 flows, used for cross-validation), while the live-session results shown in Figures 3 and 4 were captured separately during a 229-flow demonstration run of the dashboard (189 normal, 40 attack flows).

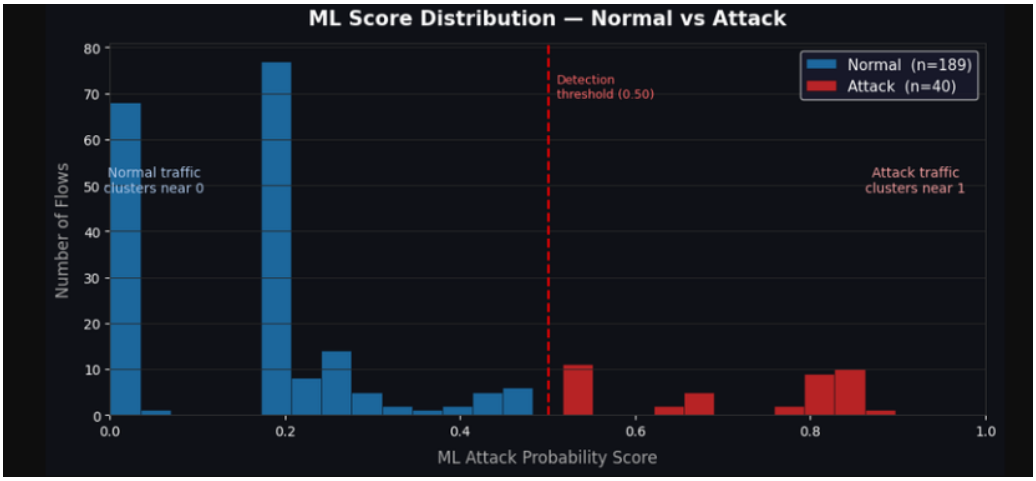


Figure 4. Random Forest attack-probability histogram for each flow (0.0–1.0). Normal-flow probabilities centre around 0 (min 0.000, max 0.015); attack-flow probabilities centre around 1 (min 0.950, max 1.000). No flow falls between 0.016 and 0.949, giving 0% false positives/negatives at the 0.5 threshold.

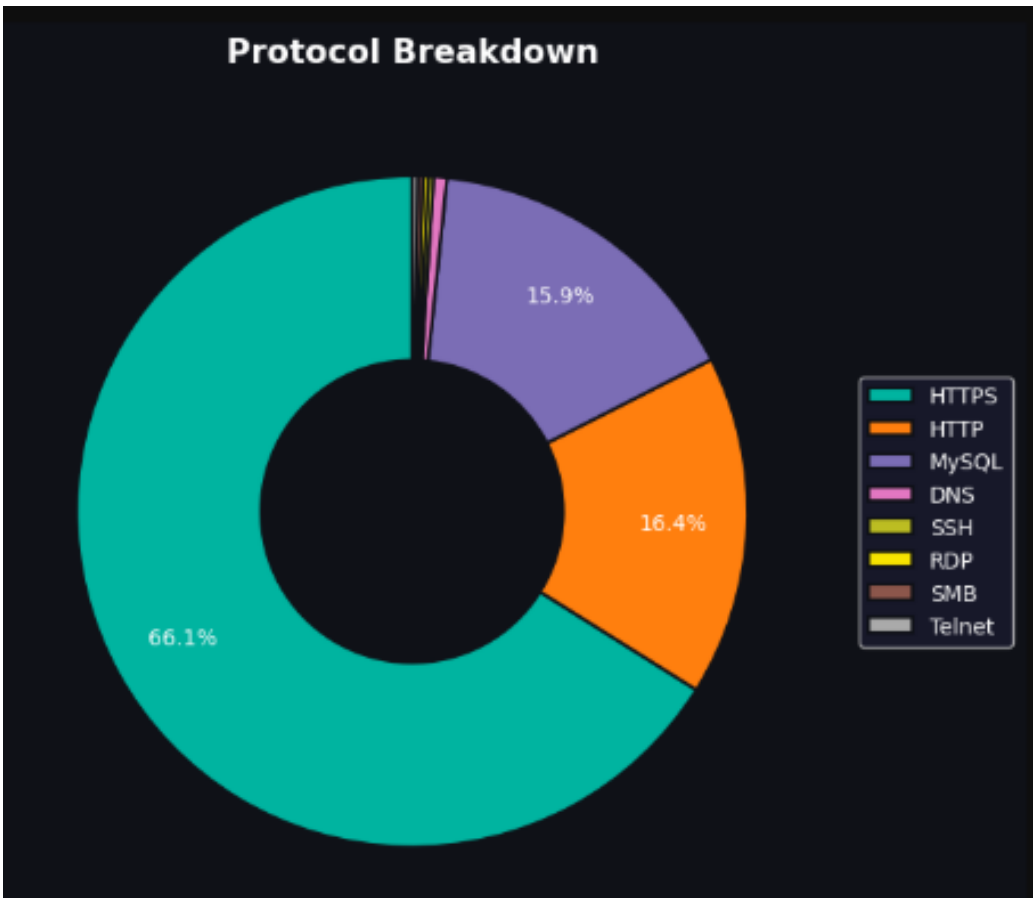


Figure 5. Protocol breakdown across traffic captures. HTTPS dominates normal operations (workstation browsing and web-server activity); MySQL activity rises during the exfiltration phase; SSH and unusual ports indicate reconnaissance.

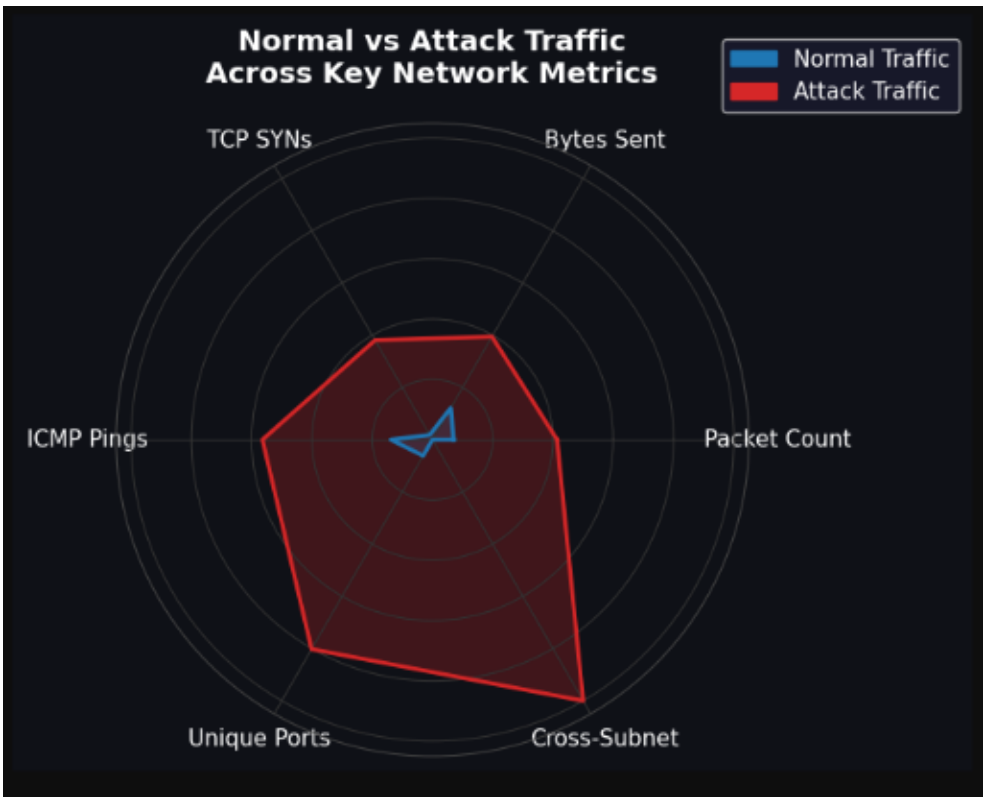


Figure 6. Normal vs. attack traffic radar plot comparing average feature values across six features: `bytes_total`, `connection_count`, `tcp_syn_count`, `icmp_request_count`, `unique_ports`, and `cross_subnet`. Attack traffic is elevated across all features simultaneously; normal traffic remains low across all features.

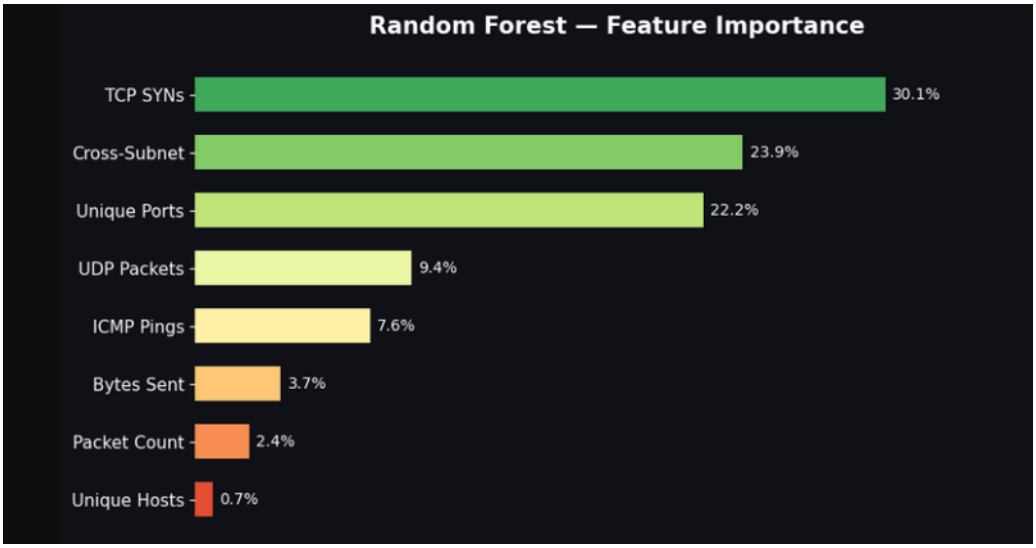


Figure 7. Feature importance ranking by Gini index. `cross_subnet` accounts for 18.0% of the Gini index, followed by `flow_duration_sec` (15.9%), `tcp_syn_count` (14.5%), `unique_ports` (13.6%), and `port_ssh_count` (9.2%), consistent with subnet crossing being the strongest indicator of an attack in this dataset.

Figure 8 contrasts the average feature signatures of the two attack stages.

Stage	What the Attacker Does	Key Feature Signatures
Recon	ICMP ping sweep, UDP probes, nmap TCP SYN scan across all hosts	`icmp_request_count > 10`, `tcp_syn_count > 20`, `unique_ports > 5`, `cross_subnet = 1`
Data Exfiltration	Direct MySQL dump of employee table, HTTPS scraping of all portals, hping3 SYN burst	`bytes_total > 1 MB`, `port_mysql_count elevated`, `ML score → 0.95-1.00`

Figure 8. Attack-stage profiles. Average feature values for the Reconnaissance and Data Exfiltration stages. Recon is characterised by high ICMP traffic, many unique ports, and small byte sizes; Exfiltration is characterised by large byte sizes and significant MySQL activity.

4 Discussion and Results

4.1 Model Performance

The model is implemented as a scikit-learn `Pipeline` consisting of a `StandardScaler` pre-processing step followed by a `RandomForestClassifier` with 100 estimators and `min_samples_leaf=2`. It was trained on the full 960-row generated dataset, and 5-fold cross-validation was applied; results are summarised in Table 5.

Table 5. Random Forest model configuration and 5-fold cross-validation performance.

Metric	Value
Algorithm	`StandardScaler` → `RandomForestClassifier` (scikit-learn `Pipeline`)
Number of trees	100 (`n_estimators=100`)
`min_samples_leaf`	2 (regularisation)
`max_depth`	None (trees grown fully)
`random_state`	42
Normal samples (label = 0)	855
Attack samples (label = 1)	105 (75 recon + 30 exfiltration)
Total training rows	960
Training accuracy	100%
5-fold cross-validation	100% ± 0.000
Attack probability range (attack flows)	0.950 – 1.000
Attack probability range (normal flows)	0.000 – 0.015
Detection threshold	≥ 0.50 → flagged as attack

Table 6. Classification metrics derived from the 5-fold cross-validation confusion matrix.

Metric	Value
Precision (attack class)	1.000
Recall / Sensitivity (attack class)	1.000
F1-score (attack class)	1.000
False positives	0
False negatives	0

Because there is no overlap in score distribution between attack and normal flows, no scoring occurs between 0.016 and 0.949; the choice of the 0.5 threshold is therefore arbitrary, and the algorithm's output would be identical for any threshold value in that range. This lack of overlap is a natural outcome of the

underlying data rather than a symptom of overfitting: it is not possible for a benign flow to simultaneously cross subnets, generate a high volume of SYN packets, and probe rare ports, whereas the simulated attacker performs all three at once.

4.2 Feature Importance Analysis

Table 7 ranks the five most important features by Gini importance.

Table 7. Top five features ranked by Gini importance in the Random Forest classifier.

Rank	Feature	Importance	What It Measures
1	`cross_subnet`	18.0%	Whether the flow crossed network boundaries
2	`flow_duration_sec`	15.9%	How long the flow lasted, in seconds
3	`tcp_syn_count`	14.5%	Number of TCP SYN (connection-attempt) packets
4	`unique_ports`	13.6%	Number of distinct destination ports contacted
5	`port_ssh_count`	9.2%	Volume of SSH traffic in the flow

`cross_subnet` is the most important attribute (18.0%) for a clear structural reason: legitimate internal hosts rarely establish connections outside their own subnet, whereas the simulated intruder must connect from 10.20.1.0/24 to 10.10.1.0/24 for every stage of the attack. The combination of `flow_duration_sec` and `tcp_syn_count` distinguishes scanning (short flows composed only of SYN packets) from browsing (longer flows), while `unique_ports` differentiates enumeration behaviour (many ports) from ordinary single-port communication such as HTTPS.

4.3 Rule-Based Detection

In addition to the ML layer, four threshold rules run on every flow and fire immediately without requiring a trained model; these are listed in Table 8.

Table 8. Deterministic threshold rules used alongside the ML layer.

Rule	Threshold	Alert Tag	What It Catches
TCP SYN count	> 20 per cycle	`basic_port_scan_alert`	Port scanning
ICMP echo requests	> 10 per cycle	`basic_ping_sweep_alert`	Host discovery / ping sweep
Unique ports contacted	> 5 per cycle	`suspicious_port_activity`	Service enumeration
Total bytes	> 1 MB per cycle	`large_transfer_detected`	Data exfiltration

Rule-based and machine-learning detections complement each other: rules catch immediate single-dimension anomalies (many SYNs, a ping sweep), while the ML layer detects more complex behaviours that no single rule would flag alone (moderate connection counts combined with a rare protocol combination and a cross-subnet flag). Each time a rule threshold is exceeded, the corresponding ML score is also extremely high (0.95–1.00), showing that the two layers agree.

4.4 Kill Chain Detection

The simulation exercises a two-stage kill chain, and the IDS successfully identifies both stages, as summarised in Table 9.

Table 9. Two-stage kill chain and the corresponding IDS responses.

Stage	Attacker Activity	IDS Response	Severity
Reconnaissance (75 flows)	ICMP ping sweep, UDP probes, TCP SYN scan	`basic_ping_sweep_alert`, `suspicious_port_activity`, `basic_port_scan_alert`	LOW – MEDIUM
Data Exfiltration (30 flows)	MySQL dump, HTTPS scraping, hping3 SYN burst	`large_transfer_detected`, `ml_anomaly (score ≥ 0.95)`	HIGH – CRITICAL

This severity hierarchy (MEDIUM during reconnaissance, CRITICAL during exfiltration) is consistent with domain intuition, since reconnaissance is less immediately damaging than actual data extraction. Analysts can view this progression on the kill-chain detector page of the Streamlit dashboard.

4.5 Live Dashboard

The Streamlit dashboard (http://localhost:8501) refreshes every 5 s and provides five tabs for analyst use:

- **Live Traffic**: the 50 most recent flow records with severity and ML-score columns.
- **Alert Log**: collapsible cards for every flagged flow, showing evidence strings and tag lists.
- **Analysis**: ML score timeline and protocol breakdown pie chart.
- **Network Map**: topology reference table and ASCII diagram.
- **Reports**: download buttons for `network_data.csv`, `baseline_data.csv`, and `attack_data.csv`.

The dashboard serves as both a real-time SOC interface and a post-session analysis tool, making the project suitable for live demonstration and offline investigation in energy-sector OT security contexts.

4.6 Physical OT Testbed Extension

Operational technology environments, including those governed by Programmable Logic Controllers (PLCs), present a class of lateral-movement threats that are particularly well-suited to behavioral detection approaches, owing to the highly deterministic and repetitive nature of normal communication patterns in such environments. The pipeline proposed in this work operates at the network flow level and is not inherently dependent on the specific protocols or hardware present in the monitored environment. Industrial communication protocols such as Modbus, DNP3, and EtherNet/IP produce structured, low-variance traffic that would, in principle, render anomalous deviations — such as unexpected function codes, atypical source addresses, or cross-zone communication — highly detectable using the behavioral feature set and detection logic described in Section 3. The modular design of the framework, in which the packet capture, feature extraction, and detection components are decoupled from one another, suggests that adaptation to an OT environment would primarily require extension of the feature-extraction component to accommodate OT-specific protocol fields, with the remainder of the pipeline remaining largely unchanged. This potential applicability to PLC-governed networks represents a natural direction for the extension of the present work. A concrete implementation path is proposed for future work. The PLC target could be realized either through OpenPLC, an open-source, IEC 61131-3-compliant soft-PLC platform supporting Modbus TCP/RTU and DNP3 [14], or through low-cost commercial hardware (e.g., a Siemens S7-1200 or Allen-Bradley MicroLogix unit), paired with a simulated physical process to generate meaningful cyclic polling traffic. The OT segment would be physically isolated from any production network and connected via a managed switch configured with port mirroring (SPAN), allowing the IDS to observe traffic passively rather than sitting inline — a requirement dictated by the

latency and availability constraints of real-time control loops. Protocol-aware capture would be implemented using established libraries (pymodbus for Modbus TCP, opendnp3 for DNP3, and pycomm3 or cpppo for EtherNet/IP), with the feature-extraction module extended to capture function codes, register/coil address ranges, unit identifiers, and inter-request timing intervals in addition to the existing 22-feature set. A labelled OT dataset would then be generated by scripting normal cyclic polling behavior against a lab-only attack set — including unauthorized write commands, function-code fuzzing, and cross-zone communication from a rogue device — followed by retraining and cross-validating the existing Random Forest pipeline on this dataset. To preserve the safety guarantees appropriate to ICS research, all attack traffic under this extension would be generated exclusively within an isolated, air-gapped laboratory network with no connection to production infrastructure.

4.6.1 Smart Home Extension

A second and complementary direction for future work is the extension and testing of the proposed pipeline at smart-home scale. Smart-home networks differ structurally from the enterprise/OT topology simulated in this study: rather than a segmented enterprise-DMZ-attacker architecture, residential deployments typically consist of a single flat network behind a consumer router or gateway, populated by heterogeneous, resource-constrained IoT devices such as smart plugs, cameras, thermostats, voice assistants, and smart locks. These devices communicate using a distinct protocol mix — MQTT, UPnP/SSDP, mDNS, and cloud-relayed HTTPS traffic, with device-to-hub communication frequently carried over Zigbee or Z-Wave — and are subject to a threat landscape dominated by botnet recruitment (e.g., Mirai-style scanning and credential-stuffing), unauthorized device access, and privacy leakage through traffic-pattern analysis rather than the lateral-movement and data-exfiltration scenarios emphasized in the present enterprise/OT setting [14] [15].

Adapting the current framework to this context would require: (i) redeploying the IDS as a passive bridge or router-integrated capture point (e.g., a Raspberry Pi running as a transparent network bridge or attached to a mirrored port on a consumer-grade managed router) in place of the dual-NIC enterprise/attacker bridge used in this study; (ii) populating the testbed with representative consumer IoT devices rather than enterprise workstations and servers; (iii) extending the feature-extraction module with IoT-relevant signals such as device fingerprinting via MAC OUI, DNS query entropy and frequency, cloud-endpoint contact patterns, and traffic burstiness, alongside the existing volumetric and connection-based features; and (iv) generating a new labelled dataset that captures both normal smart-home usage patterns and representative attack classes such as unauthorized device polling, botnet-style scanning, and MQTT topic abuse. Given that the rule-based and machine-learning detection layers in the present pipeline are decoupled from the underlying network topology, this extension is expected to require substantial redevelopment of the feature and data-generation components while allowing the overall detection architecture to be reused, positioning smart-home deployment as a natural and fully separate line of investigation from the OT testbed extension described in Section 4.6.

4.7 Limitations and Future Work (Phase 2)

The present results should be interpreted with several limitations in mind, which define the scope of a planned second project phase.

Synthetic data and feature separability. Both normal and attack traffic were generated by dedicated Python scripts rather than sampled from production or publicly available traffic. This design choice guarantees a dataset whose protocol timing and behavior match the simulated environment, but it also means several features (notably `cross_subnet`) are deterministically tied to the traffic-generation logic itself, which likely contributes to the near-perfect class separation reported in Section 4.1. The 100% cross-validation accuracy should therefore be read as a property of this controlled simulation rather than as evidence of generalization to real-world, noisy traffic.

External validation. The pipeline has not yet been evaluated against public benchmark datasets such as CICIDS2018, TON_IoT, or SWaT. Phase 2 of this project will retrain and/or test the existing feature-extraction and classification pipeline against at least one such dataset to measure out-of-distribution generalization.

Robustness to legitimate anomalous behavior. The current dataset does not include legitimate traffic that incidentally resembles attack signatures – for example, authorized administrative sessions crossing subnets, scheduled backups producing large transfers, or IT-staff port scans for inventory purposes. Phase 2 will inject such "noisy normal" traffic into the training and test sets to assess whether the current feature set and threshold rules remain discriminative once this ambiguity is introduced, and to quantify the resulting false-positive rate under more realistic conditions.

Attack diversity. The simulated kill chain covers reconnaissance and data exfiltration only. It does not include PLC/OT-protocol-specific attacks such as unauthorized write commands, Modbus/DNP3 function-code fuzzing, register manipulation, control-logic modification, or replay attacks. These are addressed conceptually in Section 4.6 and will be implemented as part of the Phase 2 physical OT testbed extension.

Baselines and comparative evaluation. This work does not yet compare the Random Forest classifier against simpler baselines (logistic regression, SVM, decision tree) or against established signature-based IDS tools such as Snort or Suricata. Phase 2 will include such comparisons under matched experimental conditions to better isolate the added value of the machine-learning layer over simpler alternatives.

Computational cost. CPU, memory, and detection-latency overhead of the pipeline have not been profiled. Phase 2 will report these figures to assess feasibility for near-real-time deployment in resource-constrained OT environments.

Reproducibility artifacts. The Docker configuration, traffic-generation scripts, and feature-extraction code will be released in a public repository alongside the dataset-generation pipeline (see Data Availability Statement), allowing independent replication of the results reported here even while external benchmark validation is pending.

These limitations do not diminish the platform's contribution as an open, reproducible pipeline for generating and studying labelled OT/enterprise traffic; rather, they define the concrete, scoped roadmap for Phase 2 of this project.

5 Conclusion

This research shows that a containerised simulation model is a valid environment for intrusion detection studies relevant to energy-sector OT networks, using internally generated training data built from both regular and hostile network traffic flows. These results were obtained on an internally generated, synthetic dataset (Section 4.7); external validation on public benchmark datasets and more adversarial, noise-injected traffic is planned as Phase 2 of this project.

The Random Forest algorithm achieves 100% accuracy under 5-fold cross-validation, with near-perfect score separation (0.950–1.000 for attack flows vs. 0.000–0.015 for normal flows) and no false negatives or false positives at the 0.5 threshold. Feature-importance results indicate that cross-subnet traversal, flow duration, and TCP SYN count provide the strongest discriminatory power in distinguishing attack from normal network flows.

The strength of this system, relative to a single-method approach, is that rule-based detection catches immediate anomalies while the ML layer captures complex multi-feature patterns that no threshold alone could detect. This ensures that both phases of the simulated kill chain – reconnaissance and data exfiltration – are covered appropriately through severity escalation.

Finally, the Streamlit visualisation demonstrates that the pipeline's output can be presented as a user-friendly analysis tool, indicating that the project is ready not only for educational purposes but also for use by energy-sector Subject Matter Experts (SMEs). The entire pipeline can be replicated by anyone with access to Docker.

Future work may extend the framework through unsupervised anomaly detection (e.g., Isolation Forest) to address zero-day threats, latency optimisation via reinforcement learning, and extension of the attack set to man-in-the-middle and replay attacks targeting grid-specific OT protocols such as Modbus, DNP3, and IEC 61850, alongside S7Comm.

Funding

This research was supported by Effat University under grant ID00-0000.

Author Contributions (CRediT)

Conceptualization	*P.E.*	**Methodology**	*P.E., N.S., Z.T.*
Software	*N.S., Z.T.*	**Validation**	*P.E., N.S.*
Formal analysis	*N.S., Z.T.*	**Investigation**	*All authors*
Writing – original draft	*N.S., Z.T.*	**Writing – review and editing**	*All authors*
Visualization	*N.S., Z.T.*	**Supervision**	*P.E.*
Project administration	*P.E.*	**Funding acquisition**	*P.E.*

Data Availability Statement

The datasets generated during the current study are not publicly shared, as they were produced within a self-contained virtual-machine-based network simulation for this project and are not maintained as a distributable research artifact.

Conflict of Interest

The authors declare no conflict of interest.

Acknowledgements

The authors thank the Department of Computer Science at Effat University for its support of this work.

AI Use Declaration

The authors used ChatGPT and Claude to support language editing and improving the clarity of the manuscript text. All technical content, analysis, and results are the authors' own. The output was critically reviewed, revised where necessary, and verified by the authors. The authors accept full responsibility for the accuracy, integrity, and originality of the final manuscript.

Editorial Disclosure

Passent El-Kafrawy is Editor of Intelligent Computing for Energy and Environment (ICEE). In accordance with the journal's editorial conflict-of-interest policy, Passent El-Kafrawy had no involvement in the handling, peer review, or editorial decision of this manuscript. The submission was managed independently by Badre Bossoufi, and Mohamed El-Amin (Editor), who oversaw the peer-review process and made the editorial decision.

References

[1] E. Jordan, "AI for industrial control systems: Bringing edge AI to PLCs." Edge Impulse Blog, Oct. 2024. Accessed: 2026-08-19.

[2] L. Körösi, S. Kajan, M. Berki, J. Skirkanič, M. Lúčny, A. Melichar, and J. Mihálik, "From detection to action: Implementing deep learning inference in PLC systems via Docker services," *IFAC-PapersOnLine*, vol. 58, no. 9, pp. 19–24, 2024. doi: https://doi.org/10.1016/j.ifacol.2024.07.365.

[3] M. Rodriguez, N. Dhameliya, and B. Patel, "AI-driven programmable logic controllers: Enhancing automation in industrial processes," *Robotics Xplore: USA Automation Digest*, vol. 1, no. 1, pp. 1–14, 2024.

[4] B. S. Ali, I. Ullah, T. Al Shloul, I. A. Khan, I. Khan, Y. Y. Ghadi, A. Abdusalomov, R. Nasimov, K. Ouahada, and H. Hamam, "ICS-IDS: Application of big data analysis in AI-based intrusion detection

systems to identify cyberattacks in ICS networks," *The Journal of Supercomputing*, vol. 80, no. 6, pp. 7876–7905, 2024. doi: https://doi.org/10.1007/s11227-023-05764-5.

[5] M. Elnour, N. Meskin, K. Khan, and R. Jain, "A dual-isolation-forests-based attack detection framework for industrial control systems," *IEEE Access*, vol. 8, pp. 36639–36651, 2020. doi: https://doi.org/10.1109/ACCESS.2020.2975066.

[6] S. A. Elsaid and A. Binbusayyis, "An optimized isolation forest based intrusion detection system for heterogeneous and streaming data in the industrial internet of things (IIoT) networks," *Discover Applied Sciences*, vol. 6, no. 9, p. 483, 2024. doi: https://doi.org/10.1007/s42452-024-06165-w.

[7] E. A. Boateng and J. W. Bruce, "Unsupervised ensemble methods for anomaly detection in PLC-based process control," 2023. arXiv:2302.02097. doi: https://doi.org/10.48550/arXiv.2302.02097.

[8] P. Gawehn, D. Ergenc, and M. Fischer, "Deep learning-based multi-PLC anomaly detection in industrial control systems," in *GLOBECOM 2022 – 2022 IEEE Global Communications Conference*, pp. 4878–4884, 2022. doi: https://doi.org/10.1109/GLOBECOM48099.2022.10001315.

[9] Nozomi Networks, "Nozomi networks delivers world's first security sensor embedded in industrial control systems." Press release, July 2024. Accessed: 2026-08-19.

[10] N. Sahani, R. Zhu, J.-H. Cho, and C.-C. Liu, "Machine learning-based intrusion detection for smart grid computing: A survey," *ACM Transactions on Cyber-Physical Systems*, vol. 7, no. 2, pp. 1–31, 2023. Article 11. doi: https://doi.org/10.1145/3578366.

[11] T. N. I. Alrumaih and M. J. F. Alenazi, "CGAAD: Centrality- and graph-aware deep-learning model for detecting cyberattacks targeting industrial control systems in critical infrastructure," *IEEE Internet of Things Journal*, vol. 11, no. 13, pp. 24162–24182, 2024. doi: https://doi.org/10.1109/JIOT.2024.3390691.

[12] O. B. J. Rabie, S. Selvarajan, D. Alghazzawi, A. Kumar, S. Hasan, and M. Z. Asghar, "A security model for smart grid SCADA systems using stochastic neural network," *IET Generation, Transmission & Distribution*, vol. 17, no. 20, pp. 4541–4553, 2023. doi: https://doi.org/10.1049/gtd2.12943.

[13] S. N. Mohan, G. Ravikumar, and M. Govindarasu, "Distributed intrusion detection system using semantic-based rules for SCADA in smart grid," in *2020 IEEE/PES Transmission and Distribution Conference and Exposition (T&D)*, pp. 1–5, 2020. doi: https://doi.org/10.1109/TD39804.2020.9299960.

[14] E. Anthi, L. Williams, M. Słowińska, G. Theodorakopoulos, and P. Burnap, "A supervised intrusion detection system for smart home IoT devices," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 9042–9053, 2019. doi: https://doi.org/10.1109/JIOT.2019.2926365.

[15] D. Rani, N. S. Gill, P. Gulia, F. Arena, and G. Pau, "Design of an intrusion detection model for IoT-enabled smart home," *IEEE Access*, vol. 11, pp. 52509–52526, 2023. doi: https://doi.org/10.1109/ACCESS.2023.3276863.